

**Development and Implementation of 'Echoes of Yesterday': An Interactive
2D Story-Based Game Featuring Adaptive AI-Controlled NPCs**

A Project Presented to
Mrs. Julie Betasolo of the College of Computing and Information Sciences
BOHOL ISLAND STATE UNIVERSITY
Balilihan Campus
Magsija, Balilihan, Bohol

In Partial Fulfillment
of the Requirements for the Subject
IT 224 System Analysis and Design

Bea A. Sarong

THE PROBLEM

Statement of the Problem

Story-based games have become a popular medium for delivering immersive narratives and engaging gameplay. However, many 2D games in this genre rely on fixed dialogues and predictable NPC (Non-Player Character) behaviors, limiting player interaction and replay value.

This study introduces *Echoes of Yesterday*, an interactive 2D story-based game that features adaptive AI NPCs. These characters respond dynamically to player choices, allowing for non-linear dialogues and multiple endings. Through a trust-based system, NPC behavior changes based on the player's decisions, making each playthrough unique. Developed using Ren'Py and Python, the game aims to enhance narrative depth, emotional connection, and player engagement by moving beyond static storytelling toward more interactive and responsive gameplay.

This study aims to design and develop *Echoes of Yesterday*, an interactive 2D story-based game featuring adaptive AI NPCs (Non-Player Characters) that respond dynamically to player decisions. The project addresses the need for deeper interactivity, narrative flexibility, and character engagement in visual novels. Specifically, this study seeks to answer the following questions:

1. What are the limitations of conventional NPC behavior in existing 2D story-based games in terms of dynamic interaction and player engagement?
2. How can adaptive AI be implemented to create non-linear dialogues and character responses that change based on player actions?
3. In what ways can a trust-based system affect the development of multiple story paths and endings within a 2D game framework?

Significance of the Study

The realization of this project, *Echoes of Yesterday: An Interactive 2D Story-Based Game with Adaptive AI NPCs*, will aim to enhance narrative design and character interaction in 2D games. This study will be conducted to provide a functional contribution to the field of game development and offer the following benefits:

Game Developers. This project will serve as a guide for developers who wish to implement adaptive AI in story-based games, particularly those utilizing accessible tools such as Ren'Py and Python. It will demonstrate how trust systems and dynamic NPC behavior can enhance interactivity and narrative depth.

Players and Gaming Communities. Players will experience a more engaging and personalized story through adaptive dialogues and character reactions, encouraging multiple playthroughs and deeper emotional investment in the storyline.

Educators and Independent Creators. The game will serve as a resource for teaching narrative structure, game logic, and AI behavior. It may also inspire independent storytellers and hobbyist developers to explore interactive storytelling techniques.

Future Researchers. This study may serve as a reference for future research on AI-driven character systems and branching narratives, especially within the context of 2D or indie game development.

RESEARCH METHODOLOGY

Research Design

The researchers employed a developmental research design, which emphasizes the systematic planning, creation, and assessment of innovative technological solutions. This approach was deemed appropriate for the development of “Echoes of Yesterday,” a 2D interactive story-based game featuring adaptive AI-driven non-playable characters (NPCs). The primary objective of this study was to design and implement a dynamic digital environment that responds to player choices through a trust-based interaction system, leading to multiple narrative outcomes.

The design process followed a structured methodology beginning with the identification of user requirements and the analysis of existing systems or games with similar mechanics. The development phase incorporated iterative prototyping and testing, allowing for continuous refinement of both gameplay and adaptive AI behavior. Python, in conjunction with the Ren'Py engine, served as the primary toolset for implementing the adaptive logic and story branching.

By applying this research design, the researchers aimed to address the limitations of static NPC behavior in traditional visual novels and provide a more engaging and immersive player experience. Furthermore, the study sought to demonstrate how player-driven narratives can be enhanced through trust-based interactions, ultimately leading to varied narrative paths and multiple game endings.

Research Environment and Participants

The study was conducted at Bohol Island State University – Balilihan Campus, located in Magsija, Balilihan, Bohol. The participants consisted of selected students from the Bachelor of Science in Computer Science (BSCS) and Bachelor of Science in Information Technology (BSIT) programs. These individuals were chosen based on their familiarity with interactive games and digital systems, making them suitable evaluators of the proposed game project, “Echoes of Yesterday.” Their participation was essential in assessing the game’s functionality, engagement level, and the effectiveness of the adaptive AI-driven NPC

interactions. This selection ensured relevant and insightful feedback regarding the usability, narrative flow, and overall player experience of the developed system.

Data Gathering Instruments

Interviews, surveys, and observation were utilized by the researchers to gather the necessary data related to the system requirements, user expectations, and gameplay experience. Interviews were conducted with selected participants to understand their preferences and expectations from a choice-based interactive game. Surveys, using structured questionnaires, were administered to evaluate user experience, engagement, and satisfaction with the game's features, particularly the adaptive behavior of non-playable characters (NPCs). Observations were also carried out during gameplay sessions to assess usability issues and player interaction with the game mechanics. These instruments enabled the researchers to collect comprehensive and relevant data to support the development and refinement of the proposed system.

Research Procedure

The researchers will follow the System Development Life Cycle (SDLC) using the Agile model, which supports iterative development, continuous feedback, and collaborative planning throughout the project. This approach will allow the researchers to quickly adapt to changes, prioritize user needs, and deliver a functional, engaging, and user-centered interactive story-based game. The SDLC framework will guide each phase of development to ensure that the game is well-structured, responsive, and aligned with the intended outcomes of the study.

Phase I: Requirement Gathering and Analysis

In the initial phase, the researchers will conduct interviews, surveys, and observations to collect relevant data from selected participants within the Computer Science and Information Technology programs. This data will be used to identify user expectations, gameplay preferences, and technical requirements. The researchers will also analyze existing interactive games to assess standard practices and determine areas for innovation. The outcome of this phase will be a clear set of system requirements and user needs that will guide the succeeding development stages.

Phase II: System Design

Based on the requirements gathered, the researchers will design the system's architecture and flow. This will include the design of the game's user interface, character interaction logic, branching narrative paths, and the trust-based AI system for NPC behavior. Storyboarding tools and flow diagrams will be used to visualize game scenes, dialogue structures, and the five possible endings. The design phase will ensure that the game mechanics, aesthetic elements, and adaptive features are well-integrated and consistent with the intended player experience.

Phase III: System Modeling

The researchers will model the core systems of the game, focusing on player decision tracking, NPC trust management, and dialogue branching logic. Diagrams and pseudocode will be used to represent how player choices influence outcomes and how adaptive AI behaviors will respond in real-time. This phase will help identify logical issues, allow for early user testing of key mechanics, and establish a robust model for implementation.

Phase IV: Implementation

During the implementation phase, the game will be developed using Ren'Py and Python. The core features, such as interactive storytelling, adaptive NPC dialogue, and the trust system, will be programmed based on the system design. The researchers will incorporate the five narrative endings and test various pathways to ensure consistency and responsiveness. Iterative development will be practiced, with feedback collected after each playable build to refine the mechanics and improve user experience.

Phase V: Integration and Testing

The developed components will be integrated into a complete, functional system. Thorough testing will be conducted to evaluate system performance, detect bugs, and ensure that adaptive NPC behavior responds appropriately to player actions. Usability testing sessions will also be held with selected participants

to assess the game's interface, engagement level, and decision impact. Feedback gathered during this phase will be used to implement necessary adjustments.

Phase VI: Deployment

Once the system is validated and optimized, the game will be deployed for broader user access within the academic environment. A playtesting session will be organized involving students from the Computer Science and Information Technology programs to gather final feedback on usability and narrative coherence. The game's installer and documentation will be prepared for distribution, ensuring ease of access for users and future researchers.

Phase VII: Maintenance

In the final phase, the researchers will perform routine checks and implement updates based on user feedback. Adjustments may include refining dialogue options, balancing NPC trust thresholds, and improving overall system performance. Documentation will also be maintained to support future iterations or enhancements of the system.

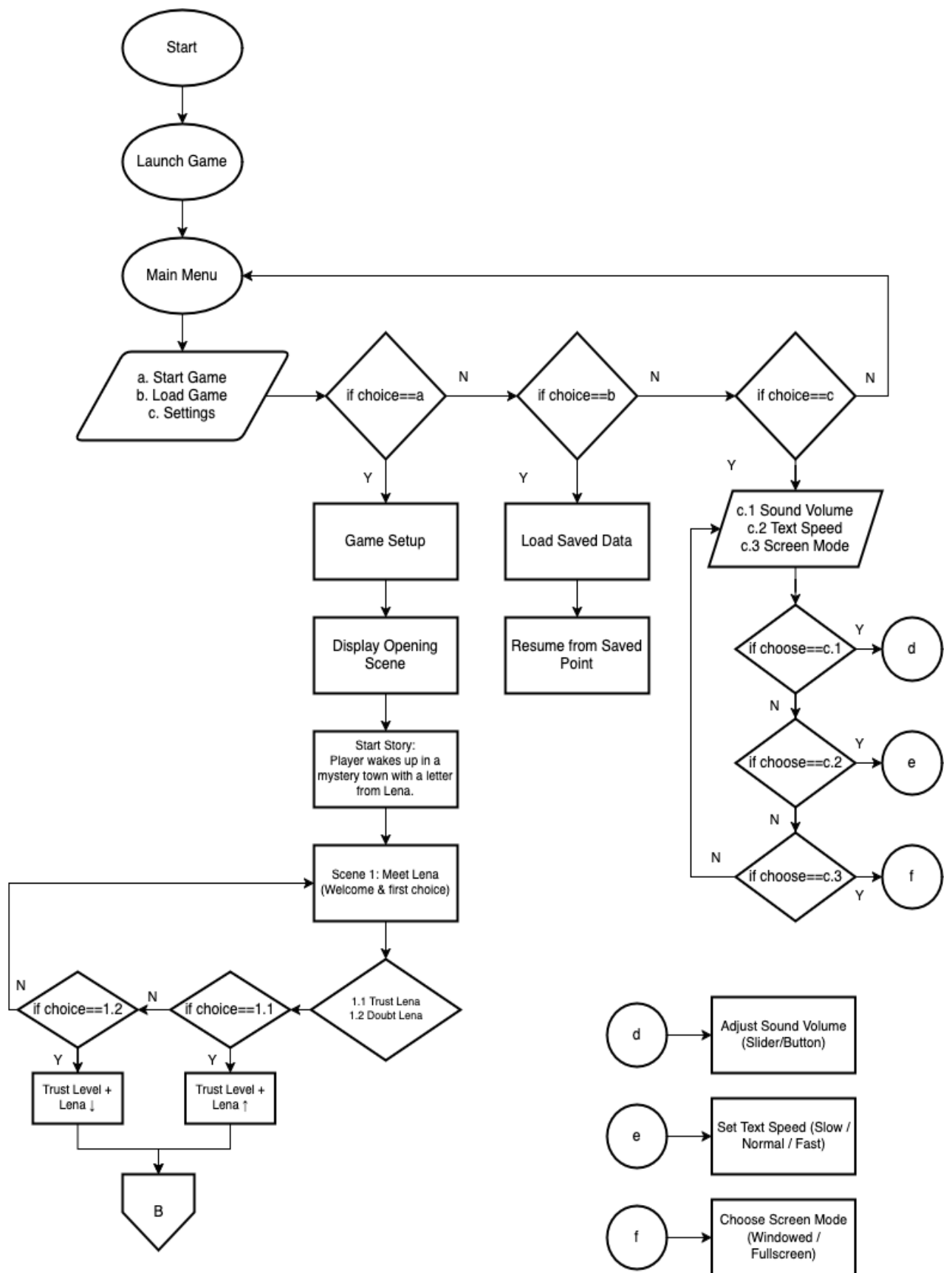


FIGURE 1: FLOWCHART GAME STORYFLOW

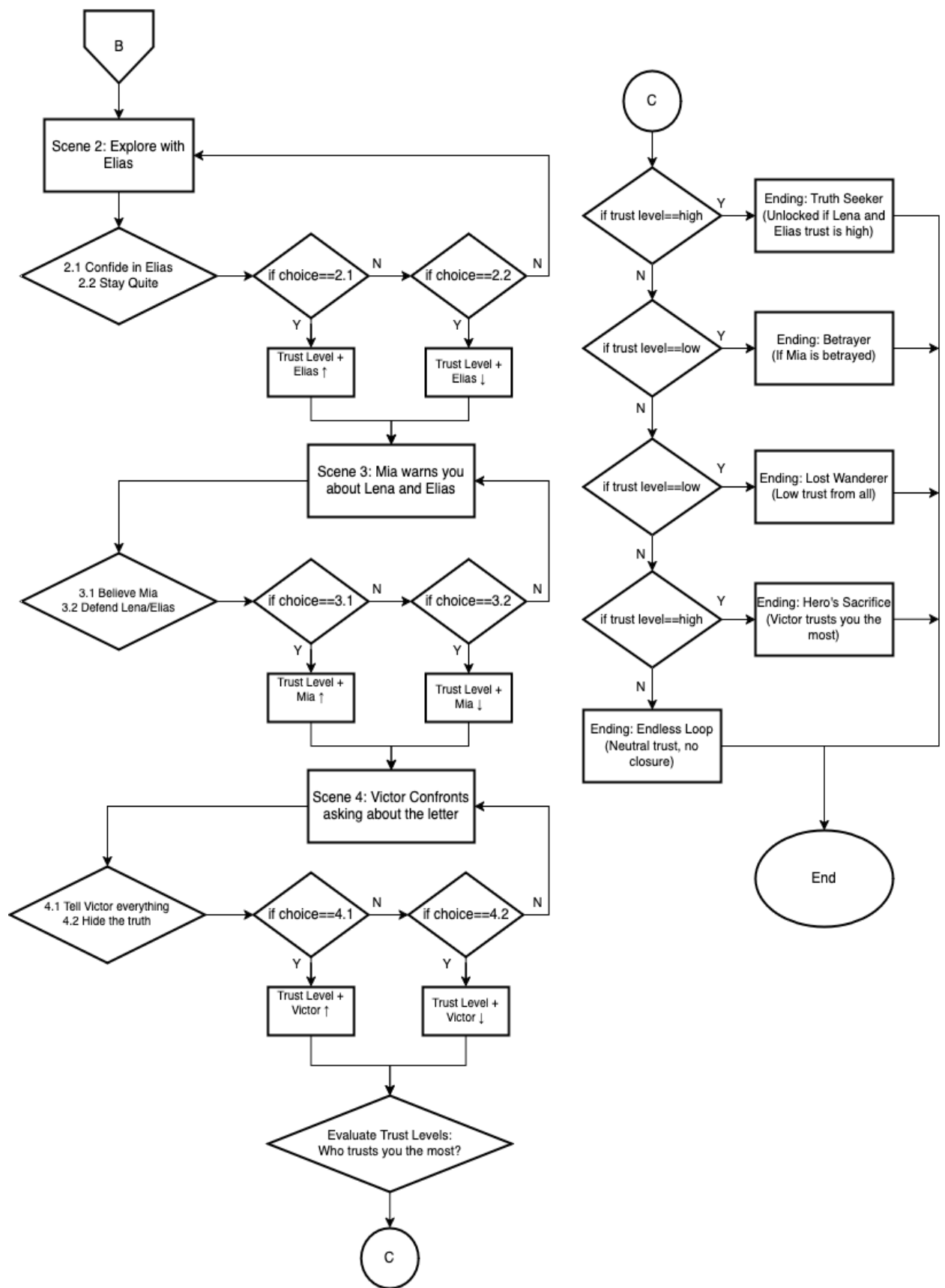


FIGURE 2: CONTINUATION

A. PRESENT SYSTEM

System Narrative

In the current landscape of 2D story-based games, a number of titles explore narrative-driven gameplay through interactive storytelling. Popular engines such as Ren'Py, RPG Maker, and Unity are widely used by independent developers to create visual novels and narrative adventures. These games typically follow a branching dialogue structure, where player choices influence character responses and plot outcomes. However, most of these systems rely on pre-scripted paths, meaning that while the player can influence outcomes, the characters themselves do not adapt dynamically to cumulative player behavior.

Adaptive AI in story-based 2D games is still in its early stages. Although some games implement basic "choice tracking" mechanics—such as increasing affinity scores with characters based on decisions—these interactions are often limited to unlocking specific endings or scenes. There is minimal use of trust systems or behavior-driven character dynamics that change dialogue and behavior in real-time, based on a player's long-term interaction history.

Furthermore, existing 2D games with branching narratives often use static dialogue trees. Once a choice is made, the system simply moves to the corresponding pre-written scene, with little to no memory of how previous choices should affect the broader game world or character relationships. As a result, many

of these games offer limited replayability, and character behavior tends to feel predictable or scripted.

Some modern games and experimental projects have begun exploring adaptive storytelling using AI and procedural narrative generation, but these are more commonly seen in 3D open-world RPGs or research prototypes and are rarely implemented in simple, accessible engines like Ren'Py. This leaves a gap in the availability of truly interactive and adaptive narrative experiences within the 2D visual novel genre.

Despite these limitations, the current systems do offer rich storytelling, multiple endings, and user engagement through emotional choices. However, there is still a need for more intelligent and immersive character systems that reflect player behavior more deeply and adapt dynamically as the story progresses.

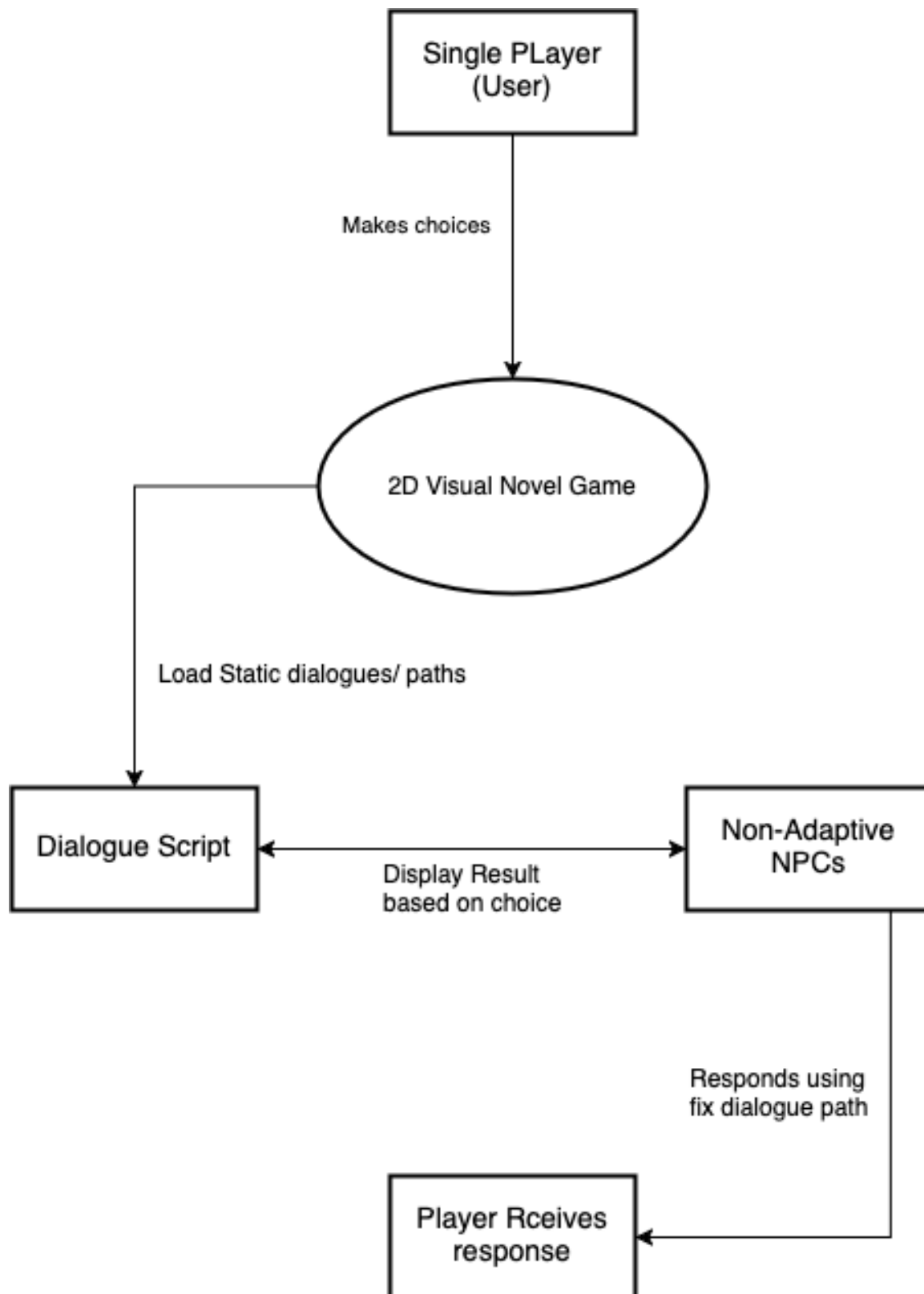


FIGURE 3: CONTEXT DIAGRAM OF THE PRESENT SYSTEM

Event List

1. The player starts the game and selects choices.
2. The game engine loads dialogue based on player choice.
3. The game engine retrieves dialogue from the script database.
4. NPCs respond using pre-written dialogue.
5. The player receives a fixed response from NPCs.

Event Diagram

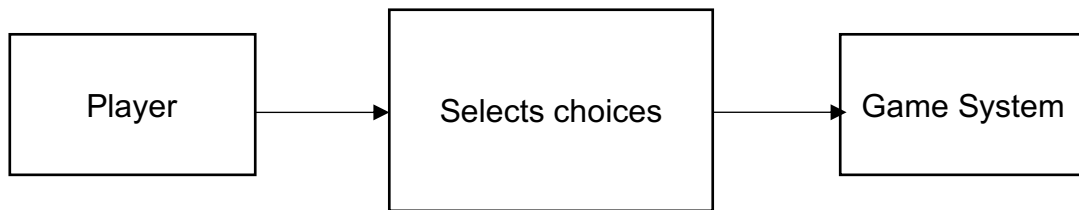


FIGURE 4: EVENT 1 SELECTS CHOICES

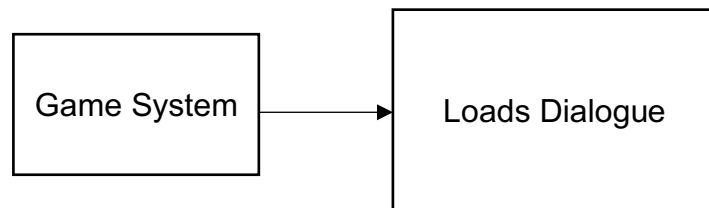


FIGURE 5: EVENT 2. LOADS DIALOGUE

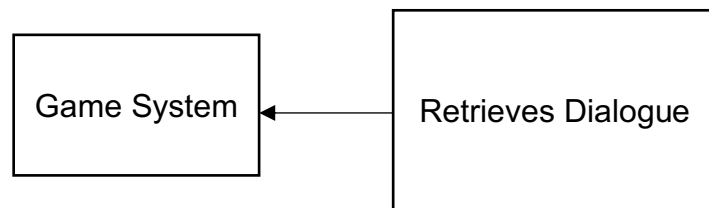


FIGURE 6: EVENT 3. RETRIEVES DIALOGUE

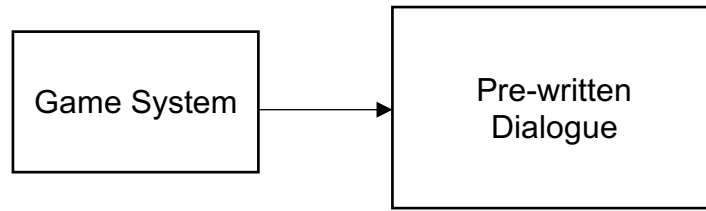


FIGURE 7: EVENT 4. PRE-WRITTEN DIALOGUE

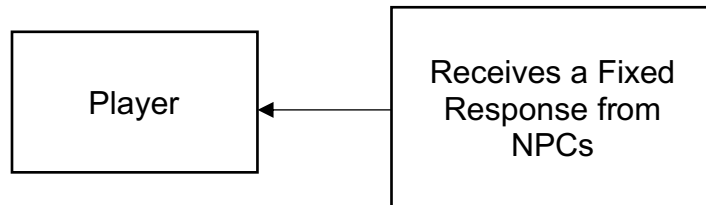


FIGURE 8: EVENT 5. RECEIVES A FIXED RESPONSE FROM NPCS

Findings and Constraints

From the analysis of the current system, the researchers identify several limitations in traditional 2D visual novel games that rely solely on static scripts. The existing setup presents challenges such as limited character interaction, repetitive dialogue experiences, and the absence of personalized responses. The system lacks adaptiveness, which affects immersion and player engagement. The identified needs and constraints of the present system are as follows:

1. The use of static dialogue paths restricts player influence on the storyline and outcomes, resulting in limited replay value.
2. Non-adaptive NPC behavior makes interactions predictable and less engaging, as all players receive the same responses regardless of their decisions.

3. The absence of a dynamic trust system prevents the game from reflecting the consequences of player choices in meaningful ways.
4. The current visual novel engines, such as Ren'Py, operate with pre-scripted branches, which do not support real-time adaptation or learning from player behavior.
5. There is no integration of AI-based decision-making, which hinders the opportunity for deeper narrative personalization and evolving NPC personalities.

These findings highlight the need for an improved system that introduces adaptive storytelling mechanics, reactive NPCs, and enhanced replayability. The limitations of the current model motivate the development of a more interactive and intelligent game structure.

B. THE PROPOSED SYSTEM

System Rules

From the analysis of the current limitations in traditional visual novel games, the researchers conceptualize *Echoes of Yesterday* as an innovative solution. The proposed system will introduce a highly interactive 2D story-based game that integrates adaptive AI-controlled NPCs to enhance narrative immersion and player engagement. At its core, the system will utilize a dynamic trust-based decision engine, enabling storylines to evolve depending on user interactions. The proposed game will combine visual storytelling, branching paths, and artificial intelligence to offer a personalized experience to every player. By introducing

adaptive mechanisms, the system will break the repetitive patterns of static scripting and elevate the player's role in shaping outcomes.

1. User Roles and Access

- **Game Player:** The end-user who will interact with the game. Players will make choices that influence the storyline, NPC behavior, and eventual game outcomes.
- **Game Developer/Admin:** The individual or team responsible for managing dialogue content, AI behavior configurations, testing, and overall system logic. This role will oversee how the game adapts to player input.

2. Dialog and Decision System

- The system will utilize dialogue trees with branching logic where each player choice will alter the current and future responses from NPCs.
- Player decisions will be tracked through a trust level system, which will be used to unlock unique dialogue paths and endings.
- NPCs will behave dynamically, meaning the same action will trigger different responses depending on previous player choices and relationship status.

3. Adaptive NPC Behavior

- The system will utilize dialogue trees with branching logic where each player choice will alter the current and future responses from NPCs.
- Player decisions will be tracked through a trust level system, which will be used to unlock unique dialogue paths and endings.
- NPCs will behave dynamically, meaning the same action will trigger different responses depending on previous player choices and relationship status.

4. Story Branching and Ending Paths

- The game will offer multiple endings, each determined by cumulative player decisions and trust levels.
- The system will calculate end conditions based on major choices, trust thresholds, and event flags encountered during gameplay.

5. Save and Load Functionality

- Players will be able to save progress at any time to revisit specific story points.
- A slot-based save system will be implemented to allow users to experiment with different outcomes by replaying previous segments.

6. Dialogue Script and Data Management

- All dialogue options, branching paths, and character data will be stored in a structured script database, allowing the system to retrieve content dynamically.
- The developer will use a centralized content management structure to update and organize the narrative content easily.

7. Security and Data Privacy

- Player data, including choices and trust scores, will be securely stored using local encryption methods.
- The system will prevent tampering or modification of story paths through access control within the game files.

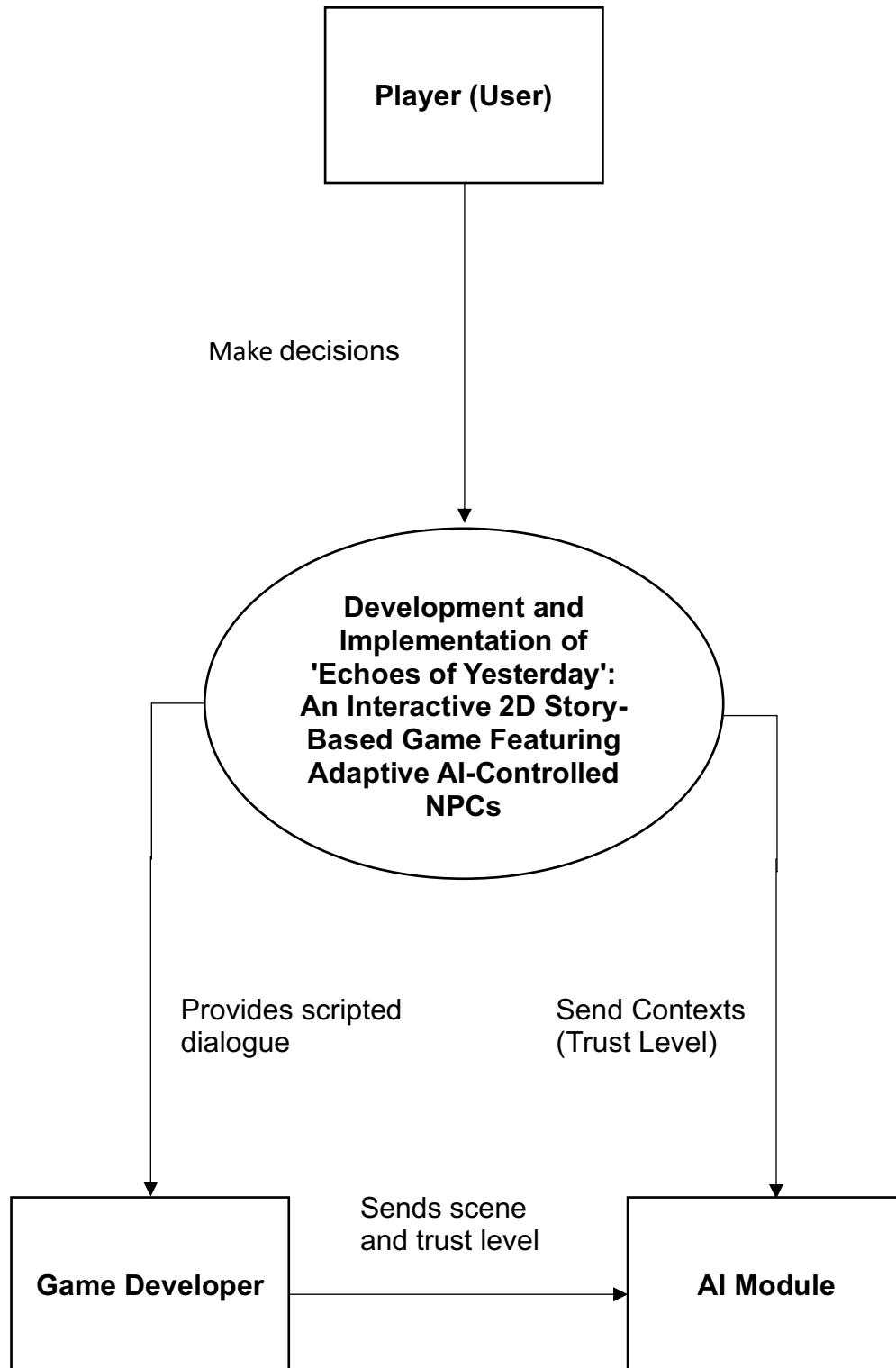


FIGURE 9: CONTEXT DIAGRAM OF THE PROPOSE SYSTEM

Event List

1. Player's Process of Making In-Game Decisions.
2. System Process of Generating NPC Dialogue.
3. System Process of Determining the Ending Based on Trust.

Event Diagram

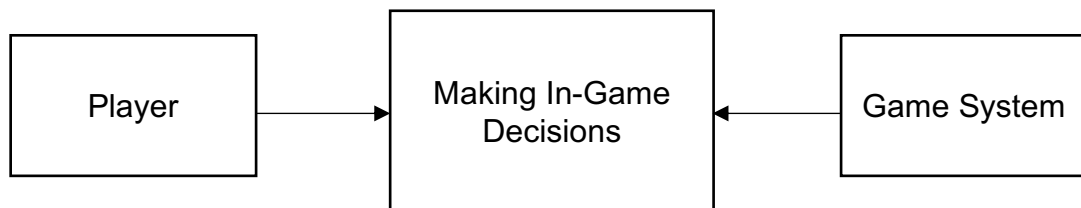


FIGURE 4: EVENT 1. MAKING IN-GAME DECISIONS

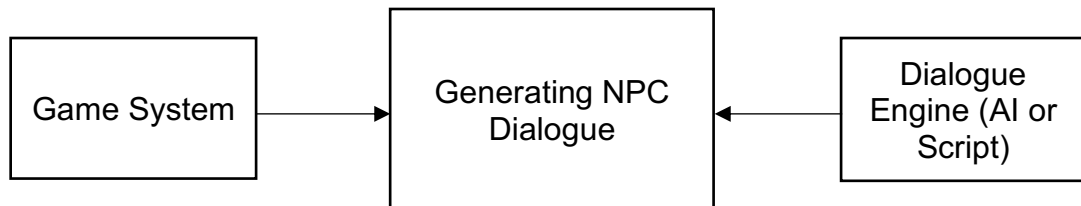


FIGURE 5: EVENT 2. GENERATING NPC DIALOGUE

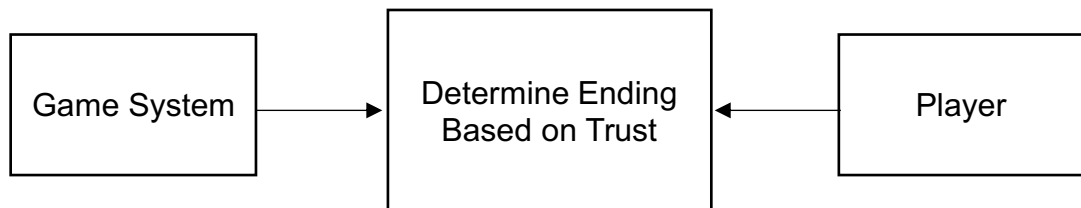


FIGURE 6: EVENT 3. DETERMINE ENDING BASED ON TRUST

Process Specification

1. Start New Game

Begin

Display game introduction and instructions

Prompt player to enter name (optional)

Load initial scene

Display message from Lena

Begin player interaction

End

2. Make In-Game Choices

Begin

Display dialogue options

Player selects choice

Record selected choice

Update NPC trust level based on choice

Display NPC response

Update current story branch

End

3. AI-Driven NPC Interaction

Begin

Player interacts with NPC

System checks NPC trust level

If trust level is high

Display positive dialogue and cooperation

Else if trust level is neutral

Display neutral dialogue

Else

Display suspicious or hostile dialogue

Adjust trust level depending on latest player action

End

4. Track and Update Trust Level

Begin

Monitor player decisions

Assign trust value per choice

Add/Subtract trust points to NPC data

Store updated trust level in system memory

Update dialogue tree based on current trust level

End

5. Branch to Storyline Events

Begin

Evaluate past player choices

Check trust level thresholds

Trigger corresponding event or cutscene

Update game flags for major decisions

Redirect to relevant story path

End

6. Determine Game Ending

Begin

Monitor all major choices and trust levels

At final chapter:

Check all decision flags

Match with ending condition

Display ending scene (e.g., Truth Seeker, Betrayer, etc.)

Display closing message and credits

End

7. Save and Load Game

Begin

Player selects "Save" or "Load" from menu

If Save:

Store current scene, choices, and trust values in save slot

If Load:

Retrieve game data from selected slot

Resume from saved point

End

8. Settings and Exit Game

Begin

Player opens settings menu

Adjust sound, text speed, or skip options

Confirm and save settings

Player selects Exit

Prompt: "Do you want to exit the game?"

If Yes:

Return to main menu or close application

End

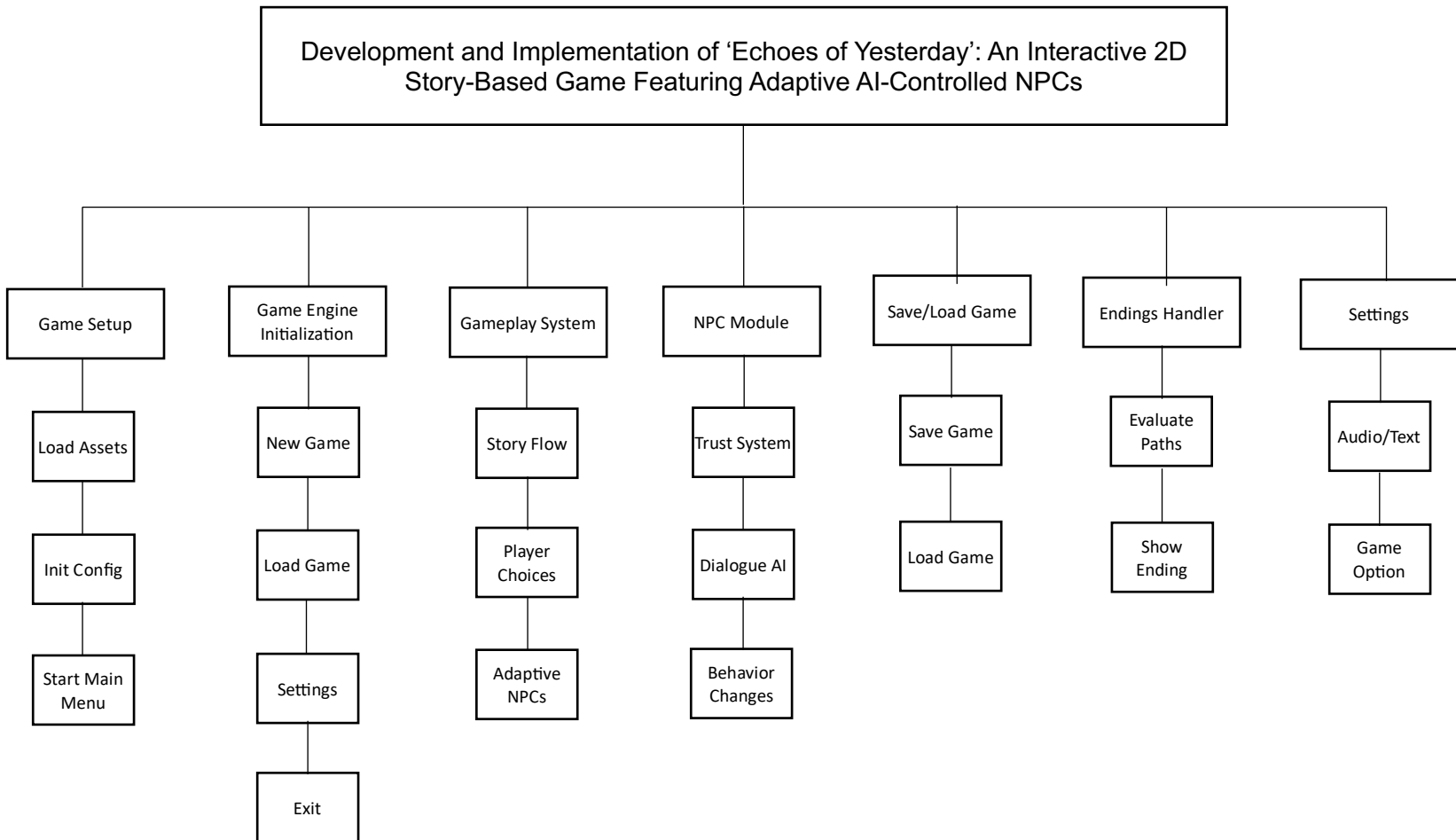


FIGURE 7: PROGRAM HIERARCHY of "Echoes of Yesterday" Game System

Database Design

The following tables are designed to support the interactive features of the "Echoes of Yesterday" game. These tables store player data such as choices, NPC relationships, and unlocked endings. This design ensures consistent tracking of game progress and personalized gameplay based on adaptive AI interactions.

Table 1. **player_choices**: A table stores all the in-game choices made by the player. It is essential in determining the flow of the story, triggering specific events or scenes, and influencing NPC behavior. It supports the dynamic and branching nature of the narrative.

Field Name	Data Type	Description
choice_id	Int	A unique identifier for each event record, serving as the primary key.
scene_id	text	Identifier or label of the scene where the choice was presented.
choice_text	text	The exact text or description of the choice selected by the player.
choice_at	timestamp	The specific date and time the choice was made during gameplay.

Table 2. **npc_trust**: A table tracks the current trust levels of each major non-playable character (NPC) in the game. Trust scores dynamically adjust based on the player's decisions, directly affecting character dialogue, actions, and story outcomes.

Field Name	Data Type	Description
npc_id	int	A unique identifier for each event record, serving as the primary key.
npc_name	varchar	The name of the NPC (e.g., Lena, Elias, Mia, Victor).
trust_score	int	A numerical value (typically from 0–100) representing the NPC's trust level.
updated_at	timestamp	The last date and time the NPC's trust score was modified.

Table 3. **unlocked_endings**: A table records the endings unlocked by the player throughout their playthrough. It is used to monitor player progress and allows for replayability tracking, achievements, or post-game summaries.

Field Name	Data Type	Description
ending_id	int	A unique identifier for each event record, serving as the primary key.
ending_title	text	The title or label of the ending (e.g., "Truth Seeker", "Lost Wanderer").
unlocked_at	timestamp	The specific date and time the ending was reached by the player.

Table 4. **scene_events**: A table contains scheduling and control data for important narrative scenes or game-triggered events. It allows the game to determine when certain story events should occur based on player progress or trust levels.

Field Name	Data Type	Description
scene_id	Int	A unique identifier for each event record, serving as the primary key.
event_name	text	The name or description of the event or scene (e.g., "Victor confrontation").
trigger_condition	text	The required condition (e.g., trust score threshold, specific choice) to trigger the event.
event_time	timestamp	The scheduled or in-game time when the event becomes available or occurs.

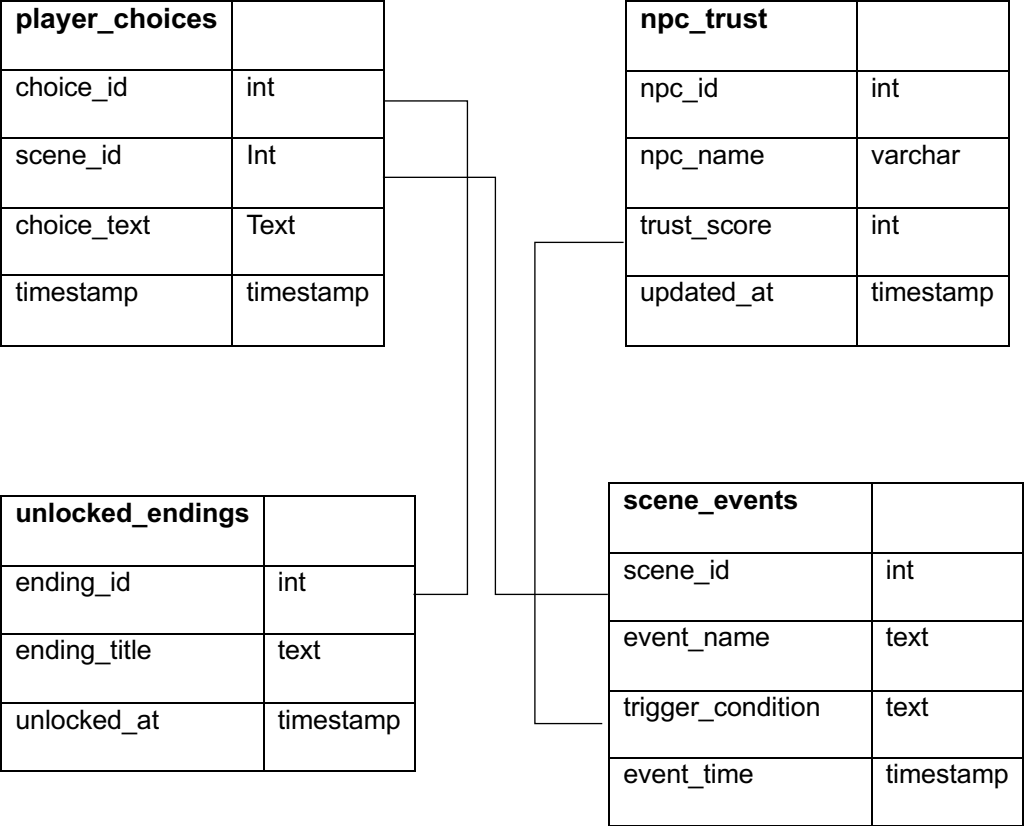


FIGURE 8: Database Schema

Test Data/Cases

1. Start New Game

INPUT: User clicks on "Start Game" from the main menu.

PROCESS:

System initializes a new player session.

Default values for trust scores and scene states are set.

Game loads introductory scene.

OUTPUT: Game begins with the first cutscene; player gains control at the first choice scene.

2. Load Save Game

INPUT: User selects a previously saved file from the "Load Game" menu.

PROCESS:

System reads save file containing last scene, choices made, trust values, and unlocked content.

Scene and game variables are restored.

OUTPUT: Game resumes at the exact saved moment with previous decisions and progress intact.

3. Adjust Settings (Volume, Text Speed, Screen Mode)

INPUT: User accesses "Settings" and changes values for volume, text speed, or screen mode.

PROCESS:

Changes are saved in game configuration.

Adjustments take effect immediately.

OUTPUT: Sound volume, dialogue speed, or screen resolution updates according to user preference.

4. Adaptive NPC Trust Behavior

INPUT: Player chooses to side with Elias during a key conflict scene.

PROCESS:

npc_trust table updates Elias's trust score (e.g., +20).

Future dialogues and interactions for Elias are conditionally altered based on updated trust value.

OUTPUT: Elias shows more openness and assists the player in a future scene not available at lower trust levels.

5. Reaching the Ending – “Truth Seeker”

INPUT: Player consistently chooses truth-seeking dialogue and helps all NPCs equally.

PROCESS:

All NPC trust scores remain moderately high.

Trigger conditions in scene_events satisfy the requirements for the "Truth Seeker" ending.

OUTPUT: Game concludes with the “Truth Seeker” ending, which is then recorded in unlocked_endings.

6. Reaching the Ending – “Betrayal”

INPUT: Player deceives NPCs and exploits key choices for personal gain.

PROCESS:

Lena's and Elias's trust scores drop below threshold.

Final confrontation scene changes based on low trust.

OUTPUT: The "Betrayer" ending is unlocked and saved in the ending log.

7. Reaching the Ending – "Lost Wanderer"

INPUT: Player avoids major story paths and fails to build trust with any NPC.

PROCESS:

Game tracks lack of key events triggered.

Fails to meet requirements for high-involvement endings.

OUTPUT: Game ends with player character wandering aimlessly—"Lost Wanderer" ending is unlocked.

8. Reaching the Ending – "Hero's Sacrifice"

INPUT: Player chooses self-sacrificial options and defends NPCs in crisis scenes.

PROCESS:

Trust levels for all NPCs spike.

A secret scene is unlocked after cumulative score reaches threshold.

OUTPUT: Game concludes with the protagonist sacrificing themselves
"Hero's Sacrifice" ending recorded.

9. Reaching the Ending – "Endless Loop"

INPUT: Player repeatedly makes paradoxical or contradictory choices across scenes.

PROCESS:

The game detects repeated contradiction flags in
player_choices.

Secret scene "Loop" is triggered.

OUTPUT: Ending scene with a time loop; "Endless Loop" is recorded in
unlocked_endings.